# 并查集 最小生成树

# I. 并查集(Union-Find Tree)

在开始正式的介绍并查集之前，我们还需要说几个图论的名词：

1. 联通图： 对于无向图来说，是任意两点有道路的图；对于无向图来说，是任意两点 u,v 可以互相到达（互相存在有向道路），这叫做有向强联通图.
2. 联通分量：即极大连通子图.
3. 邻接： 如果顶点 $u$ 和顶点 $v$ 之间存在直接相连的边，我们称这两个顶点是 邻接 的.

让我们以一个例题，来引入我们今天要讲的第一个东西吧！

对于一个具有 $N$ 个顶点的图，请快速处理以下两种操作:

- 操作 1：添加连接顶点 $u$ 和 $v$ 的双向边.
- 操作 2：判断顶点 $u$ 和 $v$ 是否属于同一联通分量.

其中，假设图初始状态无边且需要处理 $Q$ 次查询.
输入格式：第一行两个数字 $N, Q$，接下来 $Q$ 行，每行有两种情况:
对于查询一，以 1 $u$ $v$ 的形式，查询二，以 2 $u$ $v$ 的形式.
输出格式：对于每一次查询二，输出一行一个数字，表示答案.

## I.I. 什么是 Union-Find (也称为 DSU(disjoint set union))

这是一种可有效管理集合的数据结构。具体来说，可快速处理以下两种操作：

合并操作：将包含元素 $u$ 的集合和包含元素 $v$ 的集合进行合并.

查找操作：判断元素 $u$ 和元素 $v$ 是否属于同一集合。

我们发现，合并操作对应与题中的操作 1, 而查找操作则对应操作 2。

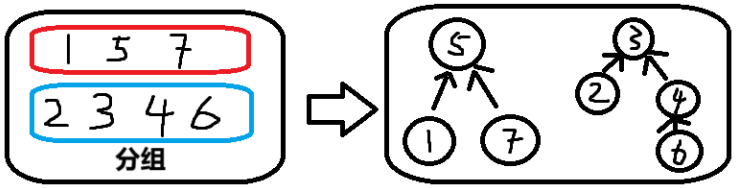
## I.II. Union-Find 的结构

那么 Union-Find 是如何处理数据的呢？简而言之，他采用满足以下条件的有根森林的可能的其中一种结构:

同一集合内的顶点在同一棵树内，有相同的根节点

不同集合的顶点在不同的树内，有不同的根节点

例如，当整数 $1-7$ 被划分为 $\{1,5,7\}$ 和 $\{2,3,4,6\}$ 两组时，其对应的 Union-Find 结构示例如下图所示.

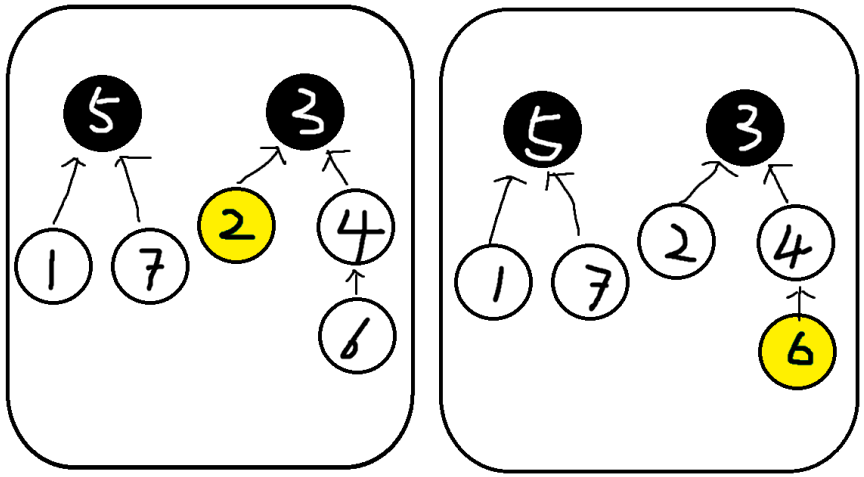


## I.III. 处理查找操作

首先介绍查找操作的处理方法。当用 $\texttt{root}(x)$ 表示顶点 $x$ 的根节点时，可通过以下方式判断元素 $u$ 与 $v$ 是否属于同一集合：

- 如果 $\texttt{root}(u) = \texttt{root}(v)$ : 同集合.
- 如果 $\texttt{root}(u) \neq \texttt{root}(v)$ : 不同集合.

在下图的案例中，元素 2 和 6 属于同一集合，因为顶点 2 和 6 的根节点均为顶点 3，即满足 $\texttt{root}(2) = \texttt{root}(6)$.




左图顶点 2 的根节点为顶点 3，右图顶点 6 的根节点为顶点 3.

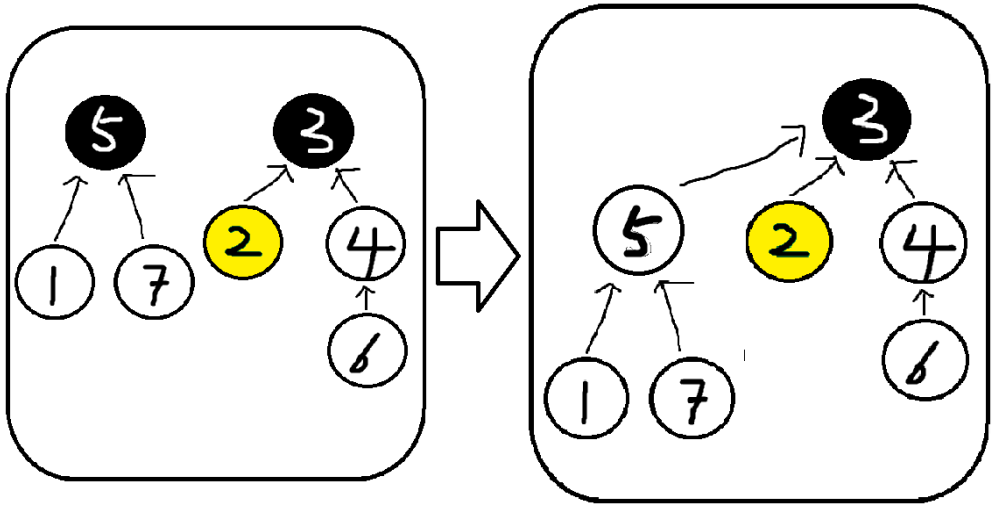
## I.IV. 处理合并操作

接下来处理合并操作，其核心操作是「连接两个数的根节点」.以下图为例，只需要将顶点 5 的父节点设为顶点 3 即可完成合并.

说的严谨一些，当用 $par[x]$ 表示顶点 $x$ 的父节点时，可通过以下步骤合并「包含顶点 $u$ 的集合」与「包含顶点 $v$ 的集合」：

- 令 $par[\texttt{root}(u)] = \texttt{root}(v)$.

也就是说，包含顶点 $v$ 的集合的根节点将成为新集合的根。当然，反向操作$par[\texttt{root}(v)] = \texttt{root}(u)$ 同样有效.



## I.V. 简单 Union-Find 的时间复杂度

简单评估一下 Union-Find 的时间复杂度. 如前所述，Union-Find 的核心是求顶点 $x$ 根节点的函数 root($x$)，其代码如下，计算次数与「顶点 $x$ 到根节点的距离」成正比！

```cpp
//返回顶点x根节点的函数，对于没有父节点的顶点，记录“par[x] = -1”
int root(int x) {
  while(true) {
    if(par[x] == -1) break; //如果没有父亲，那么这就是根节点
    x = par[x]; //前进到其父节点
  }
  return x;
}
```

我们考虑最差的情况，合并操作可能导致最长的这样的路径达到 $N-1$，此时 root($x$) 函数需要 $O(N)$ 的时间复杂度！
那么我们可不可以优化它呢？

## I.VI. 改良 Union-Find!

那么，我们考虑如何才能优化他的时间复杂度呢？

事实上，只需采用将规模更大的组置于上层的策略（被称为 启发式合并），它可以将复杂度从 $O(N)$ 降至 $O(\log N)$.

还有一种优化被称为「路径压缩」，即查询过程中经过的每个元素都属于该集合，我们可以将其直接连到根节点以加快后续查询. 同样是 $O(\log N)$.

**P.S.:**事实上，结合以上的两种写法可以将时间复杂度降低到 $O(\alpha(N))$. 其中 $\alpha(N)$ 是阿克曼函数的反函数，即使 $N = 10^5$, 他也不会超过 4.

最后 启发式合并 算法可以通过以下变量实现：

- $siz[x]$ :「以顶点 $x$ 为根的数」的顶点数.

在合并处理查询时，需要更新 $siz[x]$ 的值.

接下来给出使用 启发式合并 的代码.

```cpp
#include <bits/stdc++.h>
using namespace std;
struct UnionFind {
  int par[100009], siz[100009];
  void init(int N) {
    for(int i = 1; i <= N; i ++) par[i] = -1;
    for(int i = 1; i <= N; i ++) siz[i] = 1;
  }
  int root(int x) {
    while(true) {
      if(par[x] == -1) break;
      x = par[x];
    }
    return x;
  }
  void unite(int u, int v) {
    int RootU = root(u);
    int RootV = root(v);
    if(RootU == RootV) return;
    if(siz[RootU] < siz[RootV]) {
      par[RootU] = RootV;
      siz[RootV] = siz[RootU] + siz[RootV];
    } else {
      par[RootV] = RootU;
```

```
      siz[RootU] = siz[RootU] + siz[RootV];
    }
  }
  bool same(int u, int v) {
    if(root(u) == root(v)) return true;
    return false;
  }
};

int N, Q;
int Query[100009], u[100009], v[100009];
int main() {
  cin >> N >> Q;
  for(int i = 0; i < Q; i ++) cin >> Query[i] >> u[i] >> v[i];
  UnionFind UF;
  UF.init(N);
  for(int i = 0; i < Q; i ++) {
    if(Query[i] == 1) UF.unite(u[i], v[i]);
    if(Query[i] == 2) {
      if(UF.same(u[i], v[i]) == true) puts("Yes");
      else puts("No");
    }
  }
}
```

而如果你想要路径压缩呢？你就只需要将 root 函数修改一下：

```cpp
int root(int x) {
    if(par[x] != x) {
        par[x] = root(par[x]);
        // 路径压缩：将x的父节点直接设为根节点
    }
    return par[x];
}
```

关于这两个形式的复杂度分析，我们这里就不过多讲述了，但是前者，也就是只使用 启发式合并 的并查集复杂度，你们可以自己思考一下，为什么是 $O(\log N)$的？

我们会在将来找机会再解答这个问题！

# II. 最小生成树问题(MST)

## II.I. 什么是最小生成树？

生成树就是从 $M$ 条边中选择若干条构成的、联通所有顶点的树的结构，但同一张图可能存在两种以上的生成树方案.
其中，最小生成树是所有生成树中边权总和最小的结构.
这个定义可能比较难理解，如果将顶点视作“车站”、边视为“道路建设所需成本”，则更容易建立直观认识. 以最小成本连接所有车站的方法就是我们要找的最小生成树.
接下来我们看一个例题！

给定一个具有$N$个顶点和$M$条边的图，
顶点编号从1到$N$，
边$i$是连接顶点$A_i$和$B_i$的双向边，长度为$C_i$.
请计算该图的 最小生成树 中所有边的长度和.

请各位先行思考一会，想想怎么样可以找到最小生成树呢？
提示：可以考虑我们刚刚学过的东西！

## II.II. 求最小生成树的算法

那么如何求最小生成树呢？实际上，考虑贪心算法就可以！「优先添加最短边」的贪心算法总可以得到正确解，其具体操作如下！

1. 先将所有的边从小到大排序.
2. 每一次在最小生成树中添加当前最小的边
3. 再每一次添加边的时候，验证是否会形成一个环？如果会形成，则跳过这个边.
4. 返回第二步

那么考虑如何可以验证会不会形成环呢？
不难想到，只要使用 Union-Find ，在每一次加边的时候，判断边的两个端点是不是属于一个联通块就行！因为如果是同一个联通块，那么添加这条边就会导致环；如果不是同一个联通块，那么添加这条边就不会导致环，那么就添加这条边！
综上所述，我们就学会了最小生成树的一个重要算法 – **Kruskal** 算法！
他需要我们使用数组排序和 Union-Find 来完成该算法，复杂度是 $O(M \log M + N)$ 的.
让我们来看看代码吧！

```cpp
#include <bits/stdc++.h>
using namespace std;
int N, M;
int A[100009], B[100009], C[100009];
UnionFind UF;

int main() {
  cin >> N >> M;
  for(int i = 1; i <= M; i ++) cin >> A[i] >> B[i] >> C[i];
  vector<pair<int, int>> Edg;
  for(int i = 1; i <= M; i ++) Edg.push_back({C[i], i});
  sort(Edg.begin(), Edg.end());

  int ans = 0; UF.init();
  for(int i = 0; i < Edg.size(); i ++) {
    int idx = Edg[i].second;
    if(UF.same(A[idx], B[idx]) == false) {
      UF.unite(A[idx], B[idx]);
      ans += C[idx];
    }
  }
  cout << ans << endl;
}
```

## II.III. 思考

我们有最小生成树了！那么怎么实现最大生成树呢？
最小生成树是唯一的么？
对于一个图来说，我们要求他边权和最小的一种生成树，一定是最小生成树么？

诶，那其实这就是今天的内容了，但是各位一定想要一个练习题对吧！

2024 $ICPC$ 杭州区域赛 签到题
https://qoj.ac/problem/9726

洛谷 P1194 买礼物
https://www.luogu.com.cn/problem/P1194

其实第一道题就是并查集板子，但是有一些特判
第二道题其实就是最小生成树板子

相信各位都已经看到了上周的题单里最后有一个很奇妙的题，这里贴回来给一些没看的同学们看看:

你有 $D$ 天，每天必须从以下四个选项中选一个：
1. 花 1 日元买口香糖.
2. 花 3 日元买糖果.
3. 花 4 日元买巧克力.
4. 花 6 日元买麦麸小吃.

经过 $D$ 天后， 你一共花了 $N$ 日元. 请你计算有多少中 $D$ 天的购买序列，满足这个条件，输出答案对 998244353 取模后的结果.
$1 \le D \le 2 \times 10^5, 1 \le N \le 10^6, D, N$ 都是整数.

Atcoder FPS24 题 Snack
https://vjudge.net/problem/AtCoder-fps_24_a

# III. 生成函数入门解析

## III.I. 什么是生成函数？

生成函数就像是一个“数学工具包”，它可以把一个数列的信息打包到一个多项式中。你可以把它想象成一个特殊的多项式，其中：

- 变量的幂次：代表我们关心的条件或限制
- 系数：代表在该条件下的方案数量

简单来说，生成函数是一种形式幂级数（Formal Power Series），通过多项式的代数运算来解决计数问题。

## III.II. 具体到这道题

单天的生成函数：$A(x) = x + x^3 + x^4 + x^6$

这个式子的含义是：

- $x^1$ 的系数是 1：表示选择 1 时的方案数是 1
- $x^3$ 的系数是 1：表示选择 3 时的方案数是 1
- $x^4$ 的系数是 1：表示选择 4 时的方案数是 1
- $x^6$ 的系数是 1：表示选择 6 时的方案数是 1

**D** 天的生成函数：$F(x) = A(x)^D = \left(x + x^3 + x^4 + x^6\right)^D$

当我们需要考虑 D 天的情况时，就是把单天的生成函数自乘 D 次。展开后的$x^N$项系数就是答案。

## III.III. 关键的因式分解技巧

直接展开$\left(x + x^3 + x^4 + x^6\right)^D$很困难，但我们可以先进行因式分解：

$A(x) = x + x^3 + x^4 + x^6 = x\left(1 + x^2 + x^3 + x^5\right) = x\left(1 + x^2\right)\left(1 + x^3\right)$

因此：$F(x) = \left[x\left(1 + x^2\right)\left(1 + x^3\right)\right]^D = x^D\left(1 + x^2\right)^D\left(1 + x^3\right)^D$

## III.IV. 最终转化

我们要求的是$F(x)$中$x^N$的系数，即：$\left[x^N\right]F(x) = \left[x^N\right]x^D\left(1 + x^2\right)^D\left(1 + x^3\right)^D$

由于$x^D$的存在，这等价于求：$\left[x^{N-D}\right]\left(1 + x^2\right)^D\left(1 + x^3\right)^D$

现在我们可以使用二项式定理分别展开$\left(1 + x^2\right)^D$和$\left(1 + x^3\right)^D$，然后找出乘积中$x^{N-D}$项的系数即可。

这样就将一个复杂的多项式展开问题转化为了相对简单的二项式计算问题。

那么今天就到这里了！ 你们肯定都很期待最后一道题:

ICPC 2025 Chengdu 签到题

https://qoj.ac/contest/2567/problem/14712